

Street Directionality Problem - Report

Assignment 3: SAT Solver

Tommaso Bergonzoni

June 2026

1 Introduction

The goal of this project is to solve the Street Directionality Problem. We want to maximize the number of two-way streets turned into one-way streets. At the same time, the travel time between any two crossing cannot increase by more than $p\%$ compared to the original map.

2 General Strategy

First, the program runs the Floyd-Warshall algorithm to get the initial shortest paths. These paths give us the baseline travel times before making any changes. Then, we transform the problem into a propositional satisfiability framework and solve it using Kissat.

Since a SAT solver only answers yes-or-no questions, optimization is done using an external loop in C++. The loop starts from the maximum possible street conversions and goes down one by one. For each step, we generate a new CNF formula. To handle the $p\%$ time limit, the program checks reachability using discretized time levels. In addition, it uses a sequential counter encoding to enforce the minimum number of conversions inside the formula. The first target value that the solver finds satisfiable is our optimal solution.

3 Implementation Details

3.1 Variables

The program reads the file data from standard input and saves it into these global variables:

- `n`: An integer representing the total number of street crossings.
- `p`: An integer representing the threshold percentage for the travel time increase.
- `n_vars`: A global counter for the total number of boolean variables in the CNF formula.
- `n_clauses`: A global counter for the total number of logical clauses written to the file.
- `max_allowed_dist`: An integer showing the maximum travel time allowed under the $p\%$ limit.
- `t_matrix`: An $n \times n$ matrix with the travel times between (direct) crossings, where -1 means no street exists.
- `d_init`: An $n \times n$ matrix with the original shortest path distances between all crossings.
- `two_way_streets`: A list of pairs storing the locations $\{i, j\}$ of all original bidirectional streets.

We use these matrices to store the boolean variables of the SAT model:

- `x_edge_var[i][j]`: A matrix for the final street layout. The variable is true if cars can drive directly from crossing i to crossing j , and false otherwise.
- `reach_var[i][j][d]`: A 3D matrix for path tracking. The variable is true if crossing j is reachable from crossing i in a total time of d or less.

3.1.1 Optimization Strategy and Cardinality Constraint

Let B be the set of pairs (i, j) representing all the original two-way streets where $i < j$.

To optimize with a SAT solver, the code loops through a target number of conversions k . The loop starts at $|B|$ and decreases. In each iteration, we add a cardinality constraint to the CNF formula saying that the active conversions must be at least k :

$$\sum_{(i,j) \in B} \text{conversion_trackers}_{(i,j)} \geq k$$

The program encodes this constraint using a sequential counter. The first target k that returns a valid SAT assignment gives us the maximum number of conversions.

3.1.2 Initial Minimum Distances

We compute the initial distances before building the formula using the Floyd-Warshall algorithm. I kept this approach to stay consistent with the previous assignments. It lets us reuse the same matrix logic to calculate the limits for the reachability matrix.

3.1.3 Constraints

The model applies the constraints of the problem using these CNF clauses:

- **Street Consistency**: For every two-way street, we add a clause ensuring that we do not close both directions. We also use XOR clauses to set our tracker variables to true only when exactly one direction is turned off.
- **Base Reachability** ($d = 0$): At time zero, every crossing can reach itself, but it cannot reach any other crossing. We set these values directly to true or false in the formula.
- **Reachability Propagation**: For each time step $d \geq 1$, we update the paths. If a crossing was already reached at time $d - 1$, it stays reached at time d . Also, if an intermediate node k was reached at time $d - 1$, and the street from k to j is open, then crossing j becomes reached at time d .
- **Maximum Travel Time Limit**: For every pair of crossings that were originally connected, we calculate the max time allowed using the $p\%$ limit. We then force the reachability variable at that exact time limit to be true. If the solver cannot find a path within this limit, the formula becomes unsatisfiable.

4 Results

The model was successfully implemented using SAT with the Kissat solver. The program performed well. It correctly solved all test instances within the limit of 60 seconds.

The execution times are very fast, showing a massive improvement over Part 1 (CP). The CP version had extremely long execution times and failed to solve several instances (due to

time limit). In contrast, the SAT solver solved most test cases almost instantly.

Compared to Part 2 (ILP), the results are highly consistent. Both solvers found the exact same optimal number of street conversions. While ILP remains faster on the largest instances (such as `sdp.106` and `sdp.110`), SAT is still very competitive and matches ILP's efficiency on smaller instances.

All outputs strictly respect the constraints and the time threshold p . Every generated solution passed the validation checks perfectly.